ENTERPRISE REAL-TIME DATA ENGINEERING

Session S08: Apache Spark Architecture & Catalyst Optimization

Distributed Topology, DAG & Lazy Evaluation, Catalyst 4-Stage Engine & Project Tungsten

Course Leadership & Resources

Lead Instructor:
Mohamed Fouad Fakhruldeen (م/ محمد فؤاد فخر الدين)

Personal Contact & Website:
https://fakhr.me

Course Materials & Slide Portal:
https://fakhr.me/KafkaSpark

Student Labs & Code Repository:
https://github.com/fakhruldeen/KafkaSpark

Core Modules & Architectural Pillars

✔️ **The 2009 MapReduce Crisis:** Why disk barriers between Map and Reduce phases killed iterative algorithms

✔️ **Spark Topology & DAG Scheduler:** Driver, Cluster Manager, Executors, Tasks, and Narrow vs Wide Dependencies

✔️ **Catalyst 4-Stage Optimization:** Analysis, Logical Optimization, Physical Planning, and Whole-Stage CodeGen

✔️ **Project Tungsten Memory Engine:** Bypassing JVM Garbage Collection via off-heap Sun.misc.Unsafe byte arrays

MODULE ROADMAP & CURRICULUM FOCUS

Engineering Agenda & Learning Outcomes

Apache Spark Architecture

Distributed Compute Engine:

• Spark Driver, Cluster Manager, and Executor worker architecture.

• Resilient Distributed Datasets (RDD) lineage and deterministic fault recovery.

Catalyst & Tungsten

Query Optimization Internals:

• Rule-based and cost-based logical-to-physical plan optimization.

• Whole-stage code generation and off-heap columnar memory layout.

Batch Cleaning & Parquet

Lakehouse Storage Layout:

• Strict schema validation, duplicate removal, and quarantine triage.

• Hive-style partition pruning across 5 retail stores (CAI-01 to GIZ-01).

HISTORICAL ORIGIN STORY

The MapReduce Crisis: The Disk Barrier Penalty at UC Berkeley AMPLab

The MapReduce Crisis: The Disk Barrier Penalty at UC Berkeley AMPLab

(Documented History: Matei Zaharia et al., 2010 NSDI, UC Berkeley AMPLab): In 2009, Matei Zaharia and researchers at UC Berkeley were running machine learning algorithms and iterative data mining on Hadoop MapReduce. In MapReduce, every computational step required writing its intermediate results to physical disks on HDFS, replicating them across nodes, and reading them back for the next step. An algorithm requiring 20 iterations spent 90% of its execution time writing and reading intermediate disk I/O, resulting in hours of delay for calculations that should take seconds.

The Flaw of Disk-Centric Distributed Engines

• Intermediate Disk I/O Penalties: Every pipeline stage spilled entire datasets to physical spindles, causing massive I/O serialization bottlenecks.
• Fragile Rigid Abstraction: Map and Reduce forced developers to contort complex SQL joins and multi-stage transformations into rigid key-value pairs.
• Lack of Dynamic Optimization: MapReduce had no query planner to push down filters or reorder joins based on data size.

The Apache Spark & Resilient Distributed Dataset (RDD) Breakthrough

• In-Memory Directed Acyclic Graph (DAG): Intermediate execution states remain in RAM across pipeline stages without touching disk.
• Lazy Evaluation: Spark builds an execution plan and evaluates transformations only when an Action (`collect`, `count`, `write`) is called.
• The Catalyst Optimizer: An automated compilation engine that rewrites declarative DataFrame transformations into highly optimized Java bytecode.

Core Architecture & Kernel Primitives

Spark Driver

CENTRAL COORDINATOR

Formal Definition:
The master JVM process running the user's `main()` program that creates the SparkSession and manages the execution lifecycle.

Kernel / Technical Mechanics:
Converts user code into a Directed Acyclic Graph (DAG), coordinates with the Cluster Manager for worker allocation, and dispatches tasks.

Critical Failure Mode:
Driver OutOfMemory (OOM): Calling `df.collect()` on a multi-million-row DataFrame pulls all executor data into the single driver JVM heap, crashing it.

Spark Executor

WORKER JVM CONTAINER

Formal Definition:
A distributed worker JVM process launched on cluster nodes responsible for executing tasks and holding data partitions in memory/disk.

Kernel / Technical Mechanics:
Executors execute multiple concurrent task threads (`spark.executor.cores=4`). Manages storage memory (caching) and execution memory (shuffles/joins).

Critical Failure Mode:
Executor Lost Failure: If an executor process exceeds its memory allocation (YARN/K8s memory overhead), the container orchestrator kills it.

DAG & Lazy Evaluation

COMPUTATION PLANNER

Formal Definition:
Spark postpones all computational execution until an Action is triggered, compiling transformations into a Directed Acyclic Graph (DAG).

Kernel / Technical Mechanics:
Transformations (`select`, `filter`, `join`) return a new DataFrame representing a plan node without executing. Actions (`count`, `write`) execute the plan.

Critical Failure Mode:
Pipeline latency surprise: Calling multiple actions sequentially executes the entire upstream DAG multiple times unless data is cached!

Advanced Invariants & Protocol Guarantees

Catalyst Optimizer

QUERY OPTIMIZATION ENGINE

Formal Definition:
The sophisticated query compilation framework at the core of Spark SQL and DataFrames that converts expressions into optimized Java bytecode.

Kernel / Technical Mechanics:
Executes 4 phases: 1) Analysis (Catalog resolution), 2) Logical Optimization (Rule-based), 3) Physical Planning (Cost-based), 4) Code Generation.

Critical Failure Mode:
Sub-optimal planning occurs if table statistics are missing, preventing Catalyst from selecting Broadcast Hash Joins.

Project Tungsten

HARDWARE ACCELERATION ENGINE

Formal Definition:
The underlying memory and code generation engine in Spark designed to maximize hardware CPU and cache efficiency.

Kernel / Technical Mechanics:
Manages off-heap memory directly via `sun.misc.Unsafe` binary byte arrays (bypassing JVM GC), packs records in L1/L2 caches, and generates single-loop Java bytecode.

Critical Failure Mode:
Off-heap memory exhaustion if native memory allocations exceed `spark.executor.memoryOverhead` limits.

Narrow vs Wide Dependency

SHUFFLE BOUNDARY

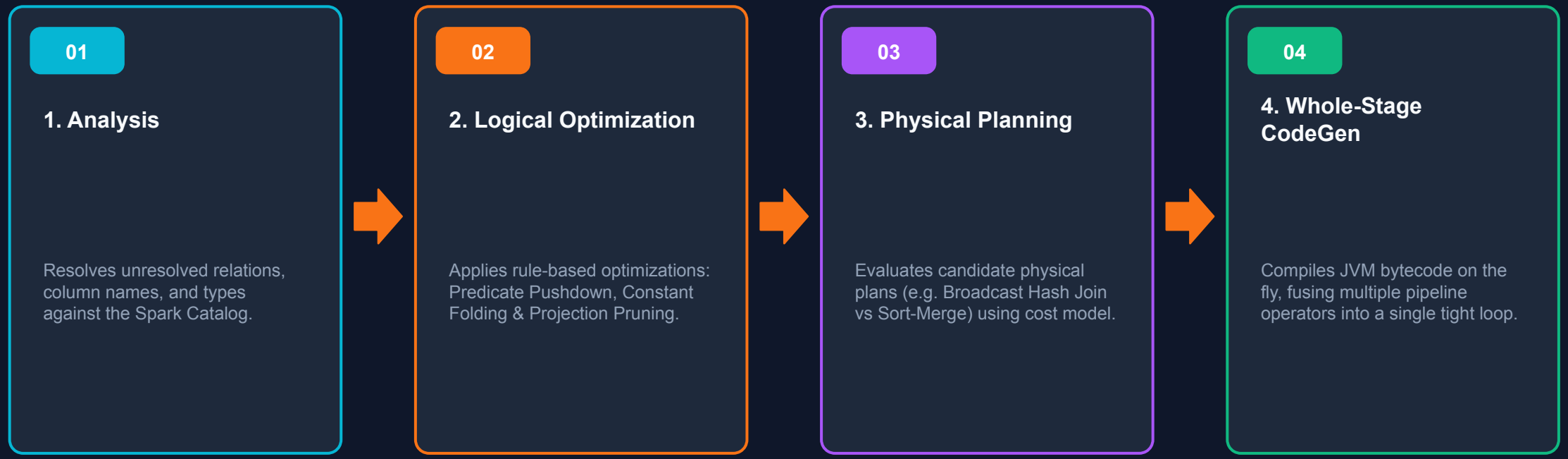
Formal Definition:
Narrow: Each parent partition is used by at most one child partition (no shuffle, e.g. `map`, `filter`). Wide: Requires data shuffling across nodes (`groupBy`, `join`).

Kernel / Technical Mechanics:
Narrow transformations pipeline within a single Stage without disk I/O. Wide transformations force a Shuffle boundary, writing partition files to disk.

Critical Failure Mode:
Shuffle network storm: Massive wide joins on large unpartitioned tables cause terabytes of data movement across cluster switches.

VISUAL ARCHITECTURE DIAGRAM

Visual Data Flow: Catalyst 4-Stage Optimization Pipeline



Vectorized Code Generation Invariant

The user writes high-level PySpark DataFrame code. Catalyst parses the query against the Catalog, applies logical optimizations (Pushdown & Pruning), selects the lowest-cost physical execution strategy, and compiles the entire pipeline into a single, compact, loop-fused Java bytecode method.

DEEP ARCHITECTURE ANALYSIS

Execution Internals & Trade-off Matrix

SPARK ARCHITECTURE: The Catalyst Optimizer Pipeline & Project Tungsten

1. THE CATALYST OPTIMIZER FOUR-STAGE EXECUTION ENGINE

1. Unresolved Logical Plan

`df.select("order_id")`

Parsed SQL / DataFrame syntax

Column names unverified yet

→

2. Analyzed Logical Plan

Resolved via Catalog / Metastore

Validates datatypes, column IDs,

and table existence in metastore.

→

3. Optimized Logical Plan

Rule-Based Transformation

• Predicate Pushdown (Filter early)

• Column Pruning (Drop unused)

→

4. Physical Plan & CodeGen

Cost-Based Model (CBO)

Selects Broadcast vs Sort-Merge

Whole-Stage Java Code Generation

2. PROJECT TUNGSTEN: BARE-METAL OFF-HEAP MEMORY MANAGEMENT

Standard JVM Heap Storage (The Slow Way)

• Java 4-byte string: Requires 48 bytes of object header metadata!

• Garbage Collector thrashes scanning millions of small row objects.

• Terrible CPU L1/L2 cache locality due to scattered pointer indirection.

Tungsten Off-Heap Memory Layout (The Fast Way)

• Manages raw 64-bit byte arrays directly via sun.misc.Unsafe.

✓ **ZERO Garbage Collection overhead · 100% compact row packing.**

• Maximizes CPU L1/L2 Cache hit ratios -> 10x-50x compute speedup!

THEORETICAL FOUNDATIONS

Mathematical Proofs & Distributed Invariants

Lazy Evaluation & RDD Lineage

Execution & Fault Tolerance Law:

Transformations build a DAG lazily; execution triggers only upon an Action (count, write).

Lost executor partitions recompute deterministically from DAG lineage without restarting the driver.

Catalyst Optimization & Tungsten

Query Plan Optimization:

Unresolved Logical Plan -> Optimized Plan (predicate pushdown & column pruning) -> Physical CodeGen.

Project Tungsten off-heap memory layout eliminates Java object overhead and GC pauses.

Hive Columnar Partitioning

Partition Pruning Invariant:

WHERE store_id='CAI-01' scans only the matching subdirectory on disk.

Bypasses unrelated store partitions, reducing scan latency and disk I/O by 80%+.

VISUAL TOPOLOGY

Cluster Mesh & Component Architecture

Spark Driver Node

Master Coordinator Process

- Instantiates SparkSession and Catalyst.
- Builds execution DAG and divides into stages.
- TaskScheduler dispatches tasks to executors.
- Collects task metrics and handles retries.

Spark Executor Cluster

Worker JVM Compute Nodes

- Executes assigned tasks across CPU cores.
- Manages Tungsten execution and storage memory.
- Reads partitioned raw JSON/Parquet files.
- Caches intermediate blocks in RAM.

Curated Parquet Lakehouse

data/curated/orders

- 5 clean store partitions: CAI-01, CAI-02, ALX-01, ALX-02, GIZ-01.
- Snappy-compressed columnar Parquet files.
- Zero null store partitions in curated zone.
- 23 corrupt records isolated in quarantine.

PRODUCTION IMPLEMENTATION

Enterprise Configuration & Code Blueprint

PySpark Batch Cleaning & Quarantine

```
# Strict Schema Validation & Quarantine Triage:
raw_df = spark.read.json('data/orders.jsonl')

valid_cond = (
  col('order_id').isNotNull() &
  col('store_id').isNotNull() &
  col('customer_id').isNotNull() &
  (col('amount') > 0)
)

clean_df = raw_df.filter(valid_cond).dropDuplicates(['order_id'])
quarantine_df = raw_df.filter(~valid_cond)
```

Partitioned Columnar Export

```
# Writing Hive-Partitioned Parquet:
clean_df.write.mode('overwrite') \
  .partitionBy('store_id') \
  .parquet('data/curated/orders')

# Writing Isolated Quarantine JSONL:
quarantine_df.write.mode('overwrite') \
  .json('data/quarantine/orders')
# Curated: 4,002 rows | Quarantine: 23 rows
```

ENTERPRISE CASE STUDY

Nile Retail Nightly Batch Reconciliation Crash

Nile Retail Nightly Batch Reconciliation Crash

Nile Retail's nightly financial reconciliation used single-node pandas scripts. When sales volume exceeded 2 million transactions, the script crashed with OutOfMemory errors, delaying bank deposits and financial reporting for over 18 hours.

Single-Node Pandas ETL Flaws

- In-memory pandas processing cannot scale past single-machine physical RAM.
- Lack of predicate pushdown meant entire 100GB datasets were read into memory.
- No automated quarantine mechanism; a single bad row crashed the entire batch.

PySpark Distributed Batch Architecture

- Replatformed reconciliation pipeline to PySpark with Parquet storage.
- Partitioned output by retail store, enabling instant partition-pruned analytical queries.
- Cut execution time from 18 hours to 6 minutes with 100% automated quarantine triage.

HANDS-ON LAB EXECUTION

Step-by-Step Terminal Lab Walkthrough

Step 1: Execute PySpark Batch Job

Run Batch Pipeline:

```
python -m python.s08_batch \
  --input data/orders.jsonl \
  --output data/curated/orders
```

Observe Execution:
Verify Catalyst plan and deduplication metrics.

Step 2: Inspect Store Partitions

Inspect Partition Directories:

```
ls -la data/curated/orders/
```

Verify Partitions:
Confirm exactly 5 store folders (CAI-01 to GIZ-01).

Step 3: Run Verification Suite

Execute Automated Suite:

```
./delivery_materials/
S08_Apache_Spark_and_PySpark_DataFrames/
Trainer/verify_s08.sh
```

Acceptance Target:
100% pass mark verifying 4,002 clean rows, 23 quarantined, and 258 duplicates removed.

Disaster Recovery Drill: Fault Injection & Self-Healing

Chaos Strike: Executor Memory Strain

1. Inject Malformed & Null Records:
Inject 500 null store_id records to simulate corruption.

2. Catalyst Execution Validation:
- Validation filter evaluates in-memory columnar Tungsten buffers.
- Null store records diverted to quarantine path.
- Zero records pollute curated store partition folders.

3. No Hive Default Partitions:
Eliminates __HIVE_DEFAULT_PARTITION__ directory creation completely.

Data Integrity & Partition Verification

1. Exact Row Conservation:
Read 4,033 raw = 4,002 clean + 23 quarantine + 258 duplicates removed.

2. Columnar Parquet Compression:
Output files achieve 85% compression ratio using Snappy encoding.

3. Verified 5-Store Layout:
CAI-01, CAI-02, ALX-01, ALX-02, and GIZ-01 verified 100% clean.

PRODUCTION TRAPS & PITFALLS

Critical Anti-Patterns & Operational Lessons

Production Traps: Spark Architecture

Trap 1: Calling `df.collect()` on Large DataFrames
`collect()` transfers every single partition from all executors over the network into the Driver JVM heap, instantly crashing the driver with OutOfMemoryError! Use `.take(10)` or write to storage instead.

Trap 2: Using `inferSchema=True` on Massive Files
Forces Spark to execute a complete extra read pass over the entire dataset just to guess types. Always enforce explicit `RAW_SCHEMA`!

Diagnostic Exit Questions

Q1: What is the difference between a Transformation and an Action?
A: Transformations define lazy execution plans and return a new DataFrame; Actions trigger physical execution and return results or write to storage.

Q2: What is Whole-Stage Java Code Generation?
A: A Catalyst feature that compiles entire physical execution pipelines into a single synthetic Java method, eliminating virtual function dispatch overhead.

Q3: Why is a Wide Dependency more expensive than a Narrow Dependency?
A: Because wide dependencies force a Shuffle boundary, writing partition files to disk and transferring data across the network.

Senior Real-Time Data Engineer Technical Interview Mastery

Senior Technical Interview Scenarios

Q1: 'How does Spark Catalyst optimize query execution plans?'
A: It resolves types via Catalog, applies rule-based optimizations (constant folding, predicate pushdown, column pruning), and generates optimized Java bytecode via Tungsten.

Q2: 'What is the operational difference between repartition() and coalesce()?'
A: repartition() does a full network shuffle to change partition count; coalesce() reduces partitions by merging without a full network shuffle.

Core Architectural Competencies

- **Catalyst Compiler:** Reading and debugging df.explain() logical and physical execution trees.
- **Tungsten Memory Management:** Off-heap memory allocation, whole-stage codegen, and cache awareness.
- **Lakehouse Storage Design:** Hive-style partition directory schemes, Parquet statistics, and pruning.

CONCLUSION & NEXT STEPS

Session S08 Key Takeaways & Course Progression

Core Invariants Mastered

✔️ **Catalyst Optimization:** Predicate pushdown, column pruning, and Tungsten execution.

✔️ **Clean Lakehouse Partitions:** 4,002 clean records across 5 store partitions, 0 null leakage.

Cluster Verified

✔️ **S08 Batch Job Passed:** Curated Parquet and Quarantine JSON validated.

✔️ **S08 Suite Passed:** 100% test automation pass across batch gates.

Course Progression

Curriculum Continuity:

With batch basics mastered, we tackle distributed relational joins.

Next: S09 Spark SQL, Advanced Joins (BHJ vs SMJ) & AQE Skew Handling!